

APPENDIX A

ARTIFACT APPENDIX

Artifact Overview

This artifact appendix provides instructions for reproducing the results presented in AURORA. The AURORA evaluation combines two complementary approaches: (1) theoretical analysis and (2) simulation-based analysis.

In the theoretical setting, each proposed technique is evaluated under probability distributions for route selection in mixnets and relevant system characteristics, including inter-node latency and mixnode processing capacity. In contrast, the simulation-based analysis is implemented in Python using the SimPy discrete-event simulation framework. In the simulations, a dynamic set of clients generates packets destined for a set of receivers, which traverse mixnodes according to the routing strategies considered in the paper, including RLP, REP, and RBR. Along each route, mixnodes perform mixing operations to anonymize and forward packets. The artifact contains approximately 10K lines of Python code and provides scripts and clear instructions for reproducing the experiments, figures, and tables presented in the paper. For practical artifact evaluation, the default configuration reduces the number of iterations to one over independent network snapshots while keeping all other parameters the same as in the paper.

A. Description & Requirements

How to access

The artifact is available through both a persistent Zenodo archive and the corresponding GitHub repository:

- Zenodo: <https://doi.org/10.5281/zenodo.21874304>
- GitHub: <https://github.com/AURORA-mixnet/AURORA>

The GitHub repository and Zenodo contain the same files, except that GitHub includes only the short version of the dataset (approximately 20 MB), which is sufficient for artifact reproduction. Zenodo additionally includes the larger version (approximately 3 GB) for users interested in running experiments over more iterations.

Hardware and software dependencies

The artifact can run on standard systems with 16 GB RAM and 50 GB of disk space. It was tested successfully on Google Colab and additionally on two hardware/software environments:

Environment A — Workstation.

- Operating system: Ubuntu 24.04.4 LTS
- CPU: Intel Core i9-9920X @ 3.50 GHz (one core used)
- Python: 3.12.3

The corresponding Python package versions are:

numpy	2.4.3	scipy	1.17.1
simpy	4.1.1	matplotlib	3.10.8
PuLP	3.3.0		

Environment B – Laptop.

- CPU: Intel Core i7-10850H @ 2.70 GHz (one core used)

- Ubuntu 18.04 LTS
- Python: 3.11.0

The corresponding dependency versions are:

numpy	2.4.4	scipy	1.17.1
simpy	4.1.1	matplotlib	3.10.8
PuLP	3.3.0		

All required dependencies are listed in [requirements.txt](#). Other compatible versions may also work; however, reported execution times may vary with processor performance and system load.

Benchmark: AURORA Dataset

Our evaluation relies on empirical latency, geographical-location, and node-processing-capacity information derived from the Nym network (<https://nym.com>) and measurements from the RIPE Atlas dataset. The standard GitHub artifact contains: [Nym_RIPE_dataset_short_version.pkl](#)

This reduced dataset contains four independent snapshots of the network: It1, It2, It3, and It4. Consequently, the bundled artifact supports experiments over up to four distinct network configurations. The default artifact evaluation uses `self.Iterations = 1` to minimize evaluation time. Evaluators may increase this value up to four without downloading any additional data. For users interested in longer experiments, we additionally provide: [Nym_RIPE_dataset_long_version.pkl](#)

through the Zenodo archive. The extended dataset supports up to 500 network snapshots and is not included directly in the GitHub repository because of its size. To use the extended dataset, change the dataset-loading path in [Baseline_functions.py](#) to [Nym_RIPE_dataset_long_version.pkl](#).

B. Artifact Installation & Configuration

After cloning or downloading the artifact repository, create or activate a Python environment and install the required dependencies using `pip install -r requirements.txt`.

The artifact can then be launched through its main entry point using `python3 main.py`. This command opens a description that helps the user enter the ID of the experiment they want to run. Alternatively, any experiment can be executed directly using `python3 main.py ID`, where ID is the numerical identifier listed in Table I. Generated figures are stored automatically in the [Figures](#) directory. Results corresponding to tables are printed directly in the terminal.

C. Major Claims

The major claims supported by the artifact are:

- **C1:** Figure 3 demonstrates the propagation latency incurred by AURORA. Across the evaluated settings, increasing the tuning parameter α or threshold T increases the latency.
- **C2:** Figure 4 demonstrates that increasing the corresponding tuning parameter α or threshold T consistently decreases M_{RL} , the processing load on mixnodes.

- **C3:** Figure 5 demonstrates that increasing the corresponding tuning parameter α or threshold T consistently decreases RSD, the predictability of route selection.
- **C4:** Figure 11(a) demonstrates that increasing the tuning parameter α results in a gradual increase in end-to-end mixnet communication latency.
- **C5:** Figure 11(b) demonstrates that increasing the tuning parameter α results in a decrease in SAA, thereby increasing the anonymity of client-destination sessions.

Claims **C1–C3** are supported by Experiment **E1**, while Claims **C4–C5** are supported by Experiment **E2**.

D. Evaluation

This section describes the experiments used to support the main claims. Executing the experiments generates the corresponding figures in the `Figures` directory. Individual figures and tables can additionally be regenerated independently using the numerical IDs in Table I.

Experiment E1: Figures 3–5 ≤ 45 min

Supported claims: C1, C2, and C3.

Figures 3, 4, and 5 are generated together for efficiency, as they share similar experimental configurations.

[Configuration Parameters] The parameters match those used for Figures 3–5 in the paper. In particular, the experiments use $L = 3$ mixnet layers and $W = 200$ mixnodes per layer. Threshold-based configurations use the corresponding T values reported in the paper, while α -based configurations use the corresponding α values. The number of iterations is set to `self.Iterations = 1` by default.

[Preparation and Execution] Run `python3 main.py` and enter 1 when prompted, or execute the experiment directly using `python3 main.py 1`.

[Results] The generated results are saved as `Fig_3a.png–Fig_3d.png`, `Fig_4a.png–Fig_4d.png`, and `Fig_5a.png–Fig_5d.png` in the `Figures` directory.

Experiment E2: Figure 11 ≤ 20 min

Supported claims: C4 and C5.

[Configuration Parameters] The parameters match those used for Figures 11(a) and 11(b), including $L = 3$, $W = 300$, REP routing, an average mixing delay of $\mu = 50$ ms, and `self.Iterations = 1`.

[Preparation and Execution] Run `python3 main.py` and enter 2 when prompted, or execute the experiment directly using `python3 main.py 2`.

[Results] The generated results are saved as `Fig_11a.png` and `Fig_11b.png` in the `Figures` directory.

Experiment E*: Individual Figures and Tables ≤ 2 h

This option allows evaluators to reproduce any individual figure or table presented in the main body of the paper, including results not directly linked to Claims C1–C5.

[Preparation and Execution] Refer to Table I for the corresponding numerical ID and execute the targeted experiment

TABLE I
MAPPING IDS TO EXPERIMENTS, FIGURES, AND TABLES.

Target	ID	Target	ID
Fig. 3	3	Fig. 4	4
Fig. 5	5	Fig. 8	8
Fig. 9	9	Fig. 11	11
Fig. 12	12	Table 1	100
Table 2	200	Table 3	300
Experiment E1	1	Experiment E2	2

directly using `python3 main.py ID`.

[Results] Figures are stored in the `Figures` directory. Table results are printed directly in the terminal.

E. Customizations: Parameter Settings & Execution Time

The network and simulation parameters are initialized in `Experiments.py` and do not require manual modification to reproduce the standard artifact. The only parameter changed for artifact testing is `self.Iterations = 1`, compared with 500 iterations in the paper.

Users interested in additional experiments may safely adjust selected parameters in `Experiments.py`:

- **Iterations:** number of independent network snapshots. The GitHub dataset supports up to 4 iterations, whereas the extended Zenodo dataset supports up to 500. To use more than four iterations, download `Nym_RIPE_dataset_long_version.pkl` from Zenodo and change the dataset path as described above.
- **num_targets:** number of target packets in the simulation. Values between 20 and 200 can be customized.
- **run:** duration of each simulation run. Values between approximately 0.3 and 1.0 can be customized.
- **delay1:** average delay introduced at each mixnode. Values in approximately the interval $[0.01, 0.08]$ can be customized.

Execution-Time and Configuration Note

The computational cost grows linearly with the number of iterations because each iteration evaluates an additional network snapshot. Consequently, an experiment requiring one hour for one iteration may require approximately three weeks when repeated 500 times. The reduced artifact is therefore intended to verify the implementation and reproduce the trends underlying the paper’s main claims within a practical artifact-evaluation time. Close numerical agreement with the paper requires using the same number of iterations as in the paper.

Other parameters in `Experiments.py` should not be modified unless the user is familiar with the dependencies among mixnet topology, routing algorithms, and simulation parameters. Arbitrary modifications may generate configurations that no longer correspond to those evaluated in the paper or may lead to invalid simulation behavior or execution errors. Because the configuration space is large, users are encouraged to contact the authors for guidance regarding substantial changes.